



**Politechnika
Śląska**

PROJEKT INŻYNIERSKI

Gra planszowa 2.5D z uczeniem maszynowym komputerowych graczy

Jagoda WIĘCEK

Nr albumu: 300503

Kierunek: Informatyka

Specjalność: Grafika Komputerowa i Oprogramowanie

PROWADZĄCY PRACĘ

Dr inż. Ewa Lach

KATEDRA Grafiki, Wizji Komputerowej i Systemów Cyfrowych

Wydział Automatyki, Elektroniki i Informatyki

Gliwice 2024

Tytuł pracy

Gra planszowa 2.5D z uczeniem maszynowym komputerowych graczy

Streszczenie

(Streszczenie pracy.)

Słowa kluczowe

Model, agent, unity, uczenie maszynowe

Thesis title

2.5D board game with machine learning for computer players

Abstract

(Thesis abstract.)

Key words

Model, agent, unity, machine learning

Spis treści

1	Wstęp	1
1.1	Cel pracy	1
1.2	Zakres pracy	1
1.3	Charakterystyka rozdziałów	1
2	Analiza tematu	3
2.1	Sformułowanie problemu	3
2.2	Znane rozwiązania	4
3	Wymagania i narzędzia	5
3.1	Wymagania funkcjonalne	5
3.2	Wymagania niefunkcjonalne	6
3.3	Opis narzędzi	7
3.3.1	Unity	7
3.3.2	C#	7
3.3.3	Python	7
3.3.4	ML-Agents	8
3.4	Przypadki użycia	9
4	Specyfikacja zewnętrzna	11
4.1	Wymagania sprzętowe i programowe	11
4.2	Instalacja	11
4.2.1	Unity	11
4.2.2	Python	12
4.2.3	Biblioteka ml-agents	12
4.3	Aktywacja	13
4.4	Sposób obsługi	14
4.4.1	Zasady gry	14
4.4.2	Ekran aplikacji	15
4.4.3	Rozgrywka	25

5	Specyfikacja wewnętrzna	29
5.1	Klasa Tile	29
5.2	Struktura danych	30
5.3	System przemieszczania	30
5.4	Biblioteka ML-Agents	31
5.5	Klasa PlayerIA	35
6	Weryfikacja i walidacja	37
6.1	Testowanie	37
6.2	Wykryte błędy	37
7	Podsumowanie i wnioski	39
7.1	Napotkane problemy	39
7.2	Możliwy rozwój	39
	Bibliografia	41
	Spis skrótów i symboli	45
	Źródła	47
	Lista dodatkowych plików, uzupełniających tekst pracy	49
	Spis rysunków	51
	Spis tabel	53

Rozdział 1

Wstęp

Gry planszowe są formą rozrywki dla ludzi od wieków jak i formą intelektualnego wyzwania. We współczesnych czasach znaczna część życia podlega cyfryzacji, od komunikacji do płatności bankowych. Z tego powodu również wszelkie gry planszowe są przenoszone do cyfrowego świata. Takie rozwiązanie pozwala na rozpoczęcie rozgrywki ulubionej gry planszowej w świecie online wraz z graczami z całego świata bez żadnych ograniczeń.

1.1 Cel pracy

Celem pracy jest zaimplementowanie gry planszowej 2.5D w unity. Dodatkowo w ramach gry zostanie zaimplementowany gracz komputerowy z wykorzystaniem uczenia maszynowego.

1.2 Zakres pracy

- Przegląd algorytmów uczenia maszynowego dostępnego w unity.
- Zaimplementowanie graczy komputerowych z algorytmem uczenia maszynowego oraz graczy komputerowych z predefiniowanymi ruchami.
- Zaimplementowanie gry planszowej Rummikub 2.5D w unity.
- Zaprojektowanie mapy do gry.

1.3 Charakterystyka rozdziałów

- **Wstęp** - przedstawienie celu oraz zakres pracy, określenie grup docelowych
- **Analiza tematu** - zawiera ogólną analizę wszystkich możliwych rozwiązań tego projektu.

- **Wymagania i narzędzia** - przedstawienie narzędzi, wykorzystanych przy tworzeniu projektu, wymagania funkcjonalne, wymagania нефункционалне oraz diagram UML przedstawiający przypadki użycia.
- **Specyfikacja zewnętrzna** - Przetawia minimalne wymagania sprzętowe, jakie użytkownik powinien spełniać, aby korzystać z aplikacji. Zawiera również instrukcje dotyczące instalacji, aktywacji, zasady gry oraz ogólny opis użytkowania aplikacji.
- **Specyfikacja wewnętrzna** -
- **Weryfikacja i walidacja** -
- **Podsumowanie i wnioski** -

Rozdział 2

Analiza tematu

Tematem pracy jest cyfrowa gra planszowa z wykorzystaniem algorytmów sztucznej inteligencji. Kluczowym problemem jest zaimplementowanie inteligentnego gracza, który poprzez system prób i błędów, nauczy się grać. Wybranie gry planszowej ma duże znaczenie, im bardziej skomplikowane zasady gry, tym trudniej będzie modelowi nauczyć się zasad. Do realizacji tego projektu została wybrana gra planszowa o nazwie Rummikub. Rummikub to gra planszowa, w której gracze rywalizują, układając płytki w:

- grupy - te same liczby w różnych kolorach
- serie - kolejne liczby w tym samym kolorze.

Celem graczy jest jako pierwszy wyłożyć wszystkie płytki, przy czym pierwszy ruch wymaga uzyskania układów o sumie co najmniej 30 punktów. Gra kończy się, gdy gracz wyłoży wszystkie płytki, a punkty zależą od liczby pozostałych płytek u przeciwników.

2.1 Sformułowanie problemu

Cyfrowe wersje gier planszowych często są projektowane z myślą o rozgrywkach online z innymi graczami za pośrednictwem internetu. Jednak stworzenie gry z komputerowym przeciwnikiem opartym na algorytmach nauczania maszynowego wymaga odmiennego podejścia i zastosowania innych technik programistycznych, model nie powinien się uczyć na graniu z żywymi graczami, gdyż zajęło by mu to za długo. Algorytmy AI potrzebują dużej ilości powtórzeń aby nauczyć się zasad gry. W projekcie zostanie uwzględniony jeden żywy gracz, gracze komputerowi z predefiniowanymi ruchami oraz komputerowy gracz z algorytmem nauczania maszynowego.

2.2 Znane rozwiązania

Na rynku jest wiele gier planszowych, najpopularniejsze z nich doczekały się swojej cyfrowej wersji. Są nimi między innymi:

- Monopoli - Jedna z najpopularniejszych gier planszowych. Gra oparta na finansach i rywalizacji z innymi graczami została wydana w wersji cyfrowej. Gra doczekała się również gier o charakterze monopolu, takimi jak **business tour**
- Go - Chińska gra planszowa, popularna również w Korei i Japonii. Celem gry jest zajęcie jak większej powierzchni na planszy od przeciwnika. Gra go ma swoją wersję cyfrową na stronach internetowych.
- Cantan - Strategiczna gra planszowa. Gracze mają za zadanie zyskiwać punkty zwycięstwa poprzez wykonywanie konkretnych ruchów na mapie. W grze stawia się budynki i drogi w miejscach strategicznych, co daje graczowi przewagę nad innymi. Wygrywa gracz, który uzyska ustaloną ilość punktów zwycięstwa. Gra planszowa **Cantan** ma swoją wersję cyfrową dostępną dla graczy z całego świata.
- Szachy - Jedna z najbardziej popularnych strategicznych gier planszowych. Gra, oparta na rywalizacji z drugim graczem, posiada wiele wersji cyfrowych. Szachy są nie tylko grą online opartą na grę innym użytkownikiem, ale również jednym z pierwszych gier, w jakiej zastosowano graczy komputerowych z algorytmami sztucznej inteligencji, czyniąc rozgrywkę jeszcze bardziej skomplikowaną.
- Rummikub - Wydana gra online Rummikub jest nastawiona na grę sieciową do komunikacji innych graczy z całego świata. Gra jest zaimplementowana w grafice 2D bez algorytmów sztucznej inteligencji.

Rozdział 3

Wymagania i narzędzia

W tej sekcji przedstawiono wymagania funkcjonalne i нефункционаłne projektu oraz narzędzia technologiczne i programistyczne, które zostały wykorzystane do jego realizacji. Opisane zostaną także kluczowe zasoby niezbędne do wdrożenia rozwiązania, takie jak środowiska programistyczne, biblioteki, frameworki oraz sprzęt.

3.1 Wymagania funkcjonalne

Interakcja z mapą :

- Gra musi pozwolić graczowi i agentowi na wybór płytki, którą chcą postawić bądź wejść z nią w interakcje.
- Gracz oraz agent powinni być w stanie postawić nową płytkę na mapie, przesunąć istniejącą bądź usunąć ją z mapy, jeśli jest to możliwe.
- Agent oraz gracz muszą wyłożyć serie bądź grupy płytek jak najszybciej.
- Gra musi ograniczać graczy, aby nie mogli tworzyć serii niedozwolonych w zasadach gry.

Ruchy gracza :

- Użytkownik musi mieć możliwość widzieć swoje płytki, wykladać je na mapę oraz tworzyć serie bądź grupy
- W trakcie tury gra na bieżąco weryfikuje czy daną płytkę można postawić, pod koniec tury sprawdza czy każdy wyłożony ciąg płytek posiada co najmniej 3 płytki w jednym ciągu.
- Gracz powinien móc zakończyć turę, cofnąć wszystkie akcje oraz dobrać nową płytkę.
- Użytkownik powinien móc posortować swoje płytki.

Komputerowi gracze :

- Jeśli jest to pierwsza tura gracza:
 - Sprawdza możliwość wyłożenia serii, gdzie jej suma wynosi co najmniej 30 lub więcej.
 - Sprawdza możliwość wyłożenia grupy, gdzie jej suma wynosi co najmniej 30 lub więcej.
- Gdy pierwsza tura zostanie zakończona:
 - Sprawdza możliwość wyłożenia serii i grup.
 - Jeśli to możliwe, dokłada płytki do innych grup i serii.

Tryby gry :

- Użytkownik przeciwko trzem graczom komputerowym
- Trzej gracze komputerowi przeciwko agentowi, tryb służy głównie do nauki modelu
- Użytkownik przeciwko modelowi, tryb służy głównie do testowania, czy model jest poprawnie nauczony.
- Użytkownik przeciwko dwóm graczom komputerowym i jednemu agentowi.

3.2 Wymagania нефункционалне

- **Kompatybilność :**
 - Gra musi działać na systemach Windows.
 - Rozgrywka wymaga klawiatury i myszki do sterowania.
- **Użyteczność :**
 - Interfejs użytkownika powinien być intuicyjny i zrozumiały dla nowych graczy.
 - Kolorystyka i oznaczenia płytek powinny odpowiadać klasycznym zasadom gry Rummikub.
- **Niezawodność :**
 - Gra nie zezwala graczom na wykonanie ruchów wbrew zasadom.

3.3 Opis narzędzi

W tej sekcji przedstawiono narzędzia, które zostały wykorzystane do zrealizowania projektu.

3.3.1 Unity

Środowisko gry zostało zaimplementowane z silnika Unity w najnowszej stabilnej wersji 2022.3.48f1. Unity jest wieloplatformowym silnikiem do tworzenia gier i aplikacji 3D jak i 2D stworzony przez Unity Technologies. Jest szeroko stosowany nie tylko do gier wideo, ale również do symulatorów, wizualizacji interaktywnych oraz aplikacji. Unity oferuje dużo możliwości, takie jak:

- Tworzenie modeli 2D i 3D
- Animacje i symulacja fizyki
- Renderowanie grafiki

3.3.2 C#

C# (C-Sharp) to nowoczesny, obiektowy język programowania opracowany przez firmę Microsoft. Łączy zalety języków C++ oraz Java, co czyni go uniwersalnym przy użytkowaniu. Może być wykorzystany do tworzenia gier wideo czy aplikacji mobilnych.

C# wyróżnia się następującymi cechami:

- Programowanie obiektowe.
- Zarządzanie pamięcią - posiadając funkcję garbage collector, nie trzeba martwić się o wycieki pamięci.
- Wszechstronność – umożliwia tworzenie aplikacji dla systemów Windows, macOS, Android, iOS oraz gier w środowisku Unity.

C# w projekcie został wykorzystany do zaimplementowania logiki gry, komunikacji z modelem oraz zarządzania interakcji między graczem a środowiskiem w Unity.

3.3.3 Python

Python to wszechstronny, wysokopoziomowy język programowania, głównie znany z prostoty i przejrzystości kodu. Najważniejsze cechy języka Python to:

- Bogata biblioteka standardowa – zawiera moduły do pracy z danymi, plikami, sieciami, obliczeniami naukowymi i innymi zadaniami.

- Przenośność i wszechstronność – Python działa na większości systemów operacyjnych, takich jak Windows, macOS czy Linux i jest wykorzystywany w aplikacjach desktopowych, webowych, a także w badaniach naukowych.
- Poprzez istnienie wielu zewnętrznych bibliotek, takich jak NumPy, TensorFlow, PyTorch, idealnie nadaje się do analizy danych.

Python w wersji 3.9 jest wykorzystany jako środowisko do implementacji modelu uczenia maszynowego, jest to konieczne w przypadku biblioteki ML-Agents.

3.3.4 ML-Agents

ML-Agents (Machine Learning Agents) to ,opracowany przez Unity Technologies, zestaw narzędzi do wykorzystywania technik sztucznej inteligencji i uczenia maszynowego w projektach w środowisku Unity. Narzędzie pozwala na implementację agentów, którzy mogą podejmować decyzje w zaimplementowanym środowisku.

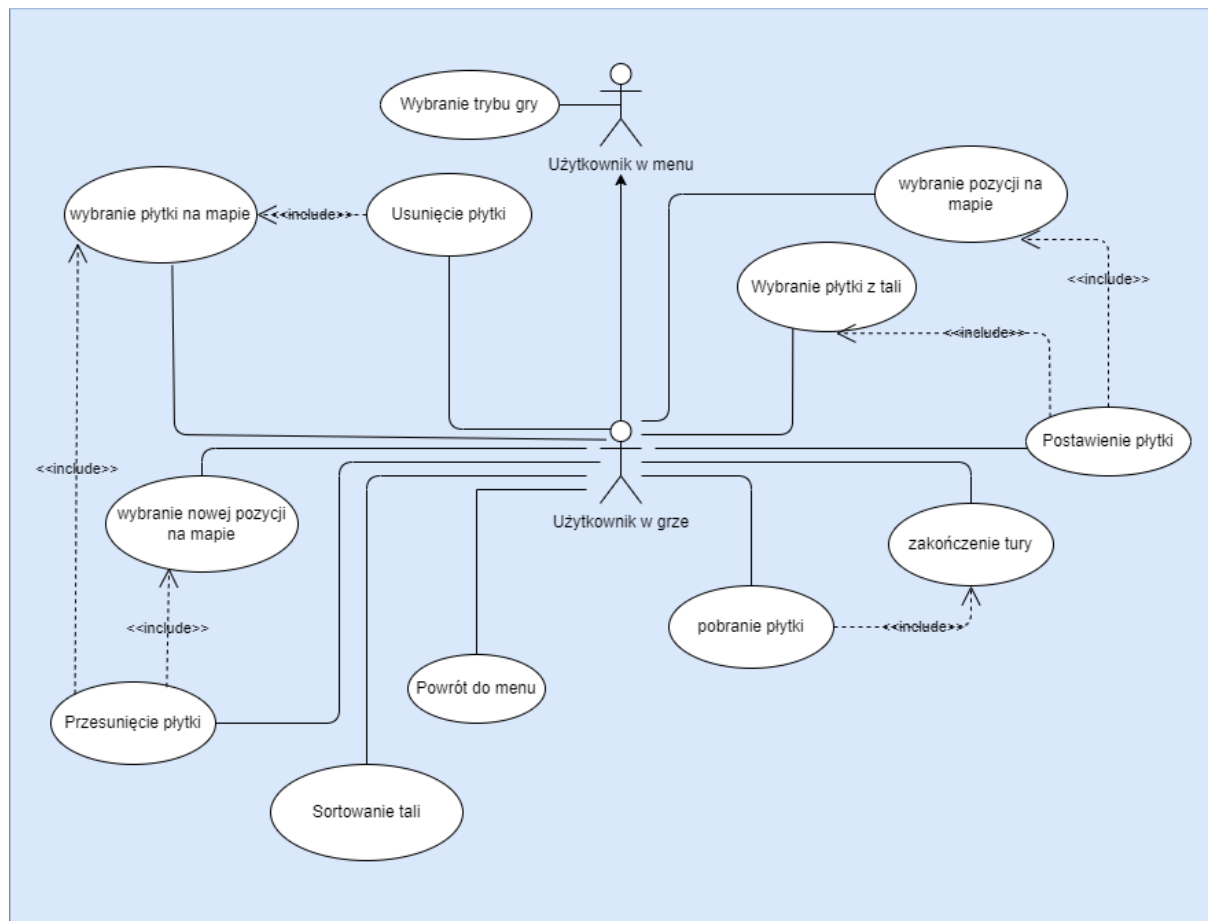
Kluczowymi cechami ML-Agents:

- Nauczanie wzmacniane - agenci uczą się na bazie dostawianych kar i nagród, jakie otrzymują.
- Narzędzie jest uniwersalne, można je zaimplementować do prostych jak i zaawansowanych gier, w których muszą podejmować decyzje.
- Agentów można zaimplementować do różnego rodzaju scenariuszy.

W tym projekcie, narzędzie, do poprawnej nauki wykorzystuje środowisko Python oraz unity. Unity jest miejscem implementacji akcji, obserwacji, nagród i kar dla modelu. Środowisko Python jest wykorzystywane do implementacji modelu oraz rozpoczynania nauki.

3.4 Przypadki użycia

Diagram przypadków użycia pokazuje możliwe funkcje dla gracza w menu oraz w grze.



Rysunek 3.1: Diagram przypadków użycia

Rozdział 4

Specyfikacja zewnętrzna

4.1 Wymagania sprzętowe i programowe

Minimalne wymagania zostały opisane w dokumentacji Unity [6], użytkownik powinien posiadać komputer z tymi bądź lepszymi komponentami:

- Windows 10 and Windows 11 tylko w wersji 64-bitowej
- CPU x64 z obsługą zestawu instrukcji SSE2.
- GPU kompatybilne z DX10, DX11 i DX12.

System był testowany na komputerze z komponentami:

- AMD Ryzen 9 7940HS w/ Radeon 780M Graphics 4.00 GHz
- 16,0 GB pamięci RAM
- System 64-bitowy Windows 11
- NVIDIA GeForce RTX 4050

4.2 Instalacja

Aby uruchomić grę w trybie edycyjnym, należy zainstalować poniższe oprogramowanie i biblioteki:

4.2.1 Unity

Do uruchomienia projektu wymagana jest instalacja Unity w wersji co najmniej 2022.3. Więcej informacji, na temat instalacji, znajduje się [3]. Aby pobrać i zainstalować Unity, należy:

1. Pobierz Unity Hub z oficjalnej strony: <https://unity.com/download>.
2. Zainstaluj Unity Hub i dodaj wersję edytora Unity 2022.3 lub nowszą.
3. Skonfiguruj środowisko projektu, upewniając się, że zainstalowane są wybrane moduły.

4.2.2 Python

Konieczne jest zainstalowanie Pythona w wersji 3.8 bądź nowsza, więcej informacji można znaleźć [1].

1. Należy pobrać instalator z odpowiednią wersją oficjalnej strony <https://www.python.org/downloads/>.
2. Uruchomić instalację.

4.2.3 Biblioteka ml-agents

Biblioteka ML-Agents Toolkit jest wymagana do wdrożenia komponentów sztucznej inteligencji w grze. Szczegółowa instrukcja instalacji znajduje się [5]. Konieczne jest zainstalowanie jej w środowisku Python jak i w Unity.

1. Przejdź do zakładki Okno (Window) w Unity
2. Wybierz Menedżer Pakietów (Package Manager)
3. W okienko wyszukiwania wpisz *ml-agents*
4. Zainstaluj wybraną wersję
5. W plikach projektu stwórz środowisko wirtualne Python przy użyciu *Wiersza poleceń*

```
python -m venv nazwa_folderu
```

bądź

```
py -m venv nazwa_folderu
```

6. Aktywuj środowisko przy użyciu:

```
nazwa_folderu\Scripts\activate
```

7. Zainstaluj w środowisku wirtualnym potrzebne biblioteki

```
py -m pip install --upgrade pip
pip install torch
```

aby zainstalować konkretną wersję biblioteki pytorch, należy wpisać komendę:

```
pip install torch==(wybrana wersja)
```

Ostatecznie należy wpisać komendę:

```
pip install mlagents
```

Jeśli ta komenda wywoła błąd, należy wpisać:

```
pip install mlagents --use-feature=2020-resolver
```

Aby zweryfikować, czy biblioteka została poprawnie zainstalowana, należy wpisać:

```
mlagents-learn --help
```

Jeśli biblioteka jest poprawnie zainstalowana, to polecenie wyświetli dostępne komendy w ramach biblioteki ml-agents.

4.3 Aktywacja

Po poprawnej instalacji i konfiguracji bibliotek oraz oprogramowania, można uruchomić aplikację.

Aby włączyć symulację aplikacji, należy wykonać następujące kroki:

1. Otwórz Unity Hub i wybierz projekt z listy w odpowiedniej wersji edytora Unity.
2. Kliknij przycisk **Open**, aby załadować projekt.
3. W edytorze Unity załaduj scenę **Menu**
4. Naciśnij przycisk **Play**, aby uruchomić grę w trybie symulacji.

Jeżeli gra została zbudowana jako aplikacja, aktywacja przebiega następująco:

1. Otwórz folder z plikami projektu (np. **Build/Release**).
2. Uruchom plik wykonywalny (na Windows jest to plik typu **.exe**).

Aby rozpocząć naukę sztucznej inteligencji (ML-Agents), należy wykonać kolejne polecenia:

1. Otwórz Unity Hub i wybierz projekt z listy w odpowiedniej wersji edytora Unity.
2. Kliknij przycisk **Open**, aby załadować projekt.
3. W edytorze Unity załaduj scenę **CP_AI_GAME**
4. W **Wierszu poleceń** przejdź do plików gry
5. Aktywuj środowisko wirtualne w Python
6. Wpisz poniższą komendę:

```
magents-learn --run-id=nazwa
```

7. Naciśnij przycisk **Play** w Unity.

4.4 Sposób obsługi

Sposób obsługi zawiera szczegółową instrukcję, jak korzystać z aplikacji z perspektywy użytkownika.

4.4.1 Zasady gry

Projekt gry Rummikub jest oparty na fizycznej grze planszowej, której zasady zostały opisane dokładnie w [2]. Gra wymaga od dwóch do czterech graczy. Gra podstawowa zawiera 106 płytek, w którym jest osiem serii o wartościach od jeden do trzynastie w czterech kolorach, oraz dwa jokery. Jokery to płytki specjalne, które mogą zastąpić każdą płytkę na mapie. Zadaniem każdego gracza jest jak najszybciej wyłożyć na stół wszystkie płytki, które zawiera w swojej tabliczce. W grze znajdują się dwa typy sekwencji płytek

- Grupy - jest to sekwencja płytek, w skład której wchodzi co najmniej trzy płytki o tej samej liczbie ale różnych kolorach. Joker może zastąpić płytkę w takiej sekwencji, grupa nie może zawierać więcej niż 4 płytki

- Serie - jest to sekwencja co najmniej trzech płytek w tym samym kolorze o różnych wartościach w kolejności od najmniejszej do największej. W takiej sekwencji może znajdować się maksymalnie 13 płytek, rozpoczynającą płytką jest zawsze o numerze jeden, a kończącą o numerze trzynaście.

Każdy gracz zaczyna rozgrywkę z czternastoma płytkami. Na początku rozgrywki gracze muszą wyłożyć sekwencje i grupy o łącznej wartości trzydziestu punktów i więcej. Każda płytka ma swoją wagę punktową, zależną od liczby wypisanej na płytce. Wartością wagową jokera jest liczba jaką zastępuje na planszy. Dopóki gracz nie wykona pierwszego ruchu, nie może manipulować żadnymi płytkami na mapie. Gracz również nie może manipulować żadnymi sekwencjami na mapie w trakcie swojego pierwszego wyłożenia. Po zakończeniu pierwszej tury, gracz może wykladać grupy i serie oraz wykonywać akcje, takie jak:

- dokładanie płytek na końcu i początku sekwencji na mapie
- rozdzielać jedną większą sekwencje na dwie mniejsze
- podmieniać jokery na mapie z płytkami ze swojego stojaka.
- przebudowywać grupy na serie oraz serie na grupy

Gracz musi w każdej turze wyłożyć co najmniej jedną płytkę, jeśli nie może tego zrobić, musi pobrać płytkę z banku, co kończy jego turę i uniemożliwia wykonanie ruchu na mapie. W grze obowiązuje limit czasu. Jeśli gracz nie zakończy swoich manipulacji przed upływem czasu, wszystkie jego ruchy należy cofnąć, a gracz dobiera płytkę z głównego banku. Gra może zakończyć się w dwóch momentach:

1. gdy jeden gracz wyłoży wszystkie płytki ze swojego stojaka
2. jeśli w głównym banku skończą się płytki a gracz, który ma swoją turę, chce dobrać kolejną.

Wynik gry zależy od płytek w stojaku gracza. Na końcu gry są podliczane wyniki na podstawie wag wartości płytek. Wagi jokerów są równe 30. Wygrywa ten, którego wynik jest najmniejszy.

4.4.2 Ekran aplikacji

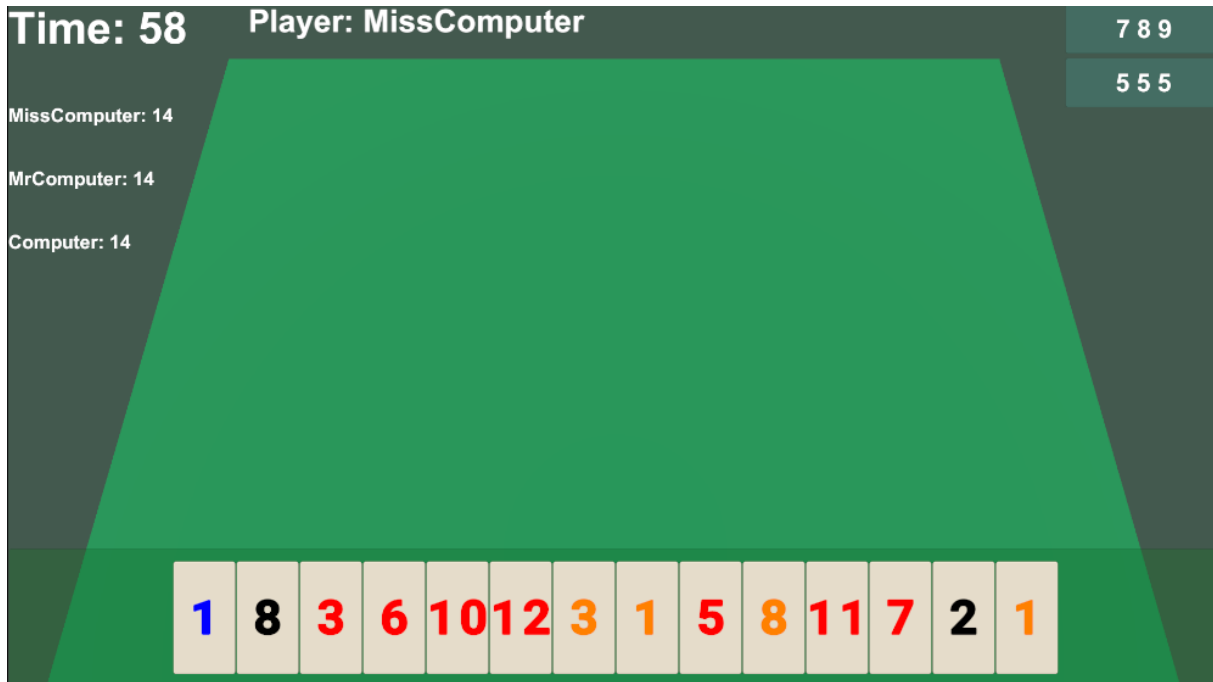
Rozgrywka rozpoczyna się widokiem menu głównego gry



Rysunek 4.1: Menu gry

Na ekranie widać cztery przyciski, każdy z nich wykonuje co innego:

- **Play with computer players** - jest to rozgrywka, w której biorą udział trzech gracze komputerowi z predefiniowanymi ruchami wraz z użytkownikiem. Tryb służy jedynie do potencjalnego zapoznania się z grą i jej mechanikami.
- **Play with AI and computer players** - tryb gry, w którym użytkownik gra wraz z dwoma graczami komputerowymi oraz jednym graczem komputerowym z algorytmami sztucznej inteligencji. Jest to docelowy tryb projektu.
- **AI vs computer players** - w tym trybie gry jest jeden gracz z algorytmami sztucznej inteligencji oraz trzech gracze komputerowi z predefiniowanymi ruchami. Ten tryb służy tylko do nauki modelu.
- **Play with AI** - ten tryb gry jest grą jeden na jeden, gdzie użytkownik gra przeciwko komputerowemu graczowi z uczeniem maszynowym. Ten tryb służy do walidacji nauki danego modelu.
- **Quit game** - przycisk służący do opuszczenia gry.



Rysunek 4.2: Przykładowy ekran początku gry

Gra rozpoczyna się z każdym graczem posiadającym po czternaście płytek na swoim stojaku.

Nad mapą widać napis:



Rysunek 4.3: Aktualna tura: Gracz X

Napis ten informuje użytkownika, który gracz ma w danej chwili wykonywać turę ruchu. W tym miejscu mogą pojawić się różne napisy:

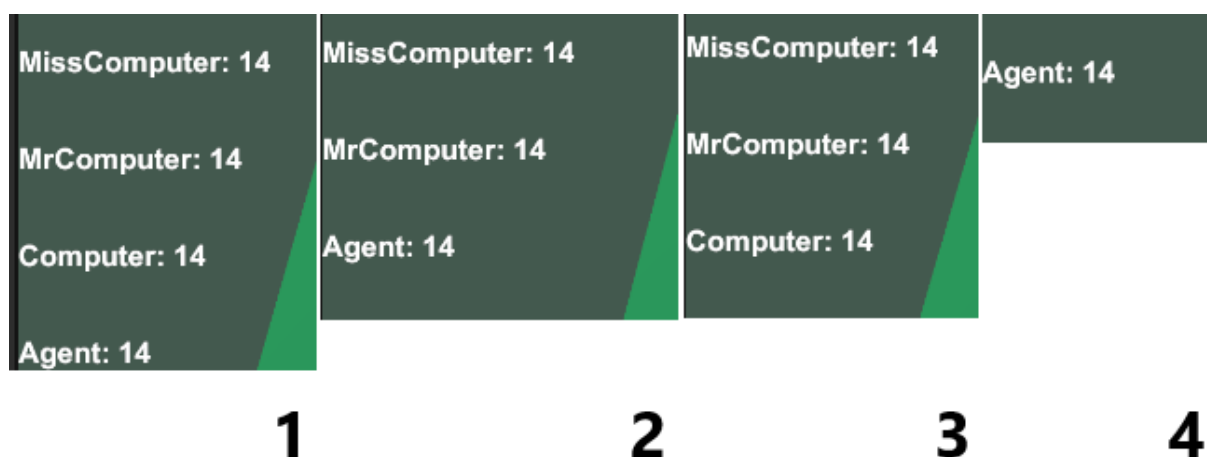
- Computer, MissComputer, MrComputer - są to wyznaczniki, że daną turę wykonuje gracz komputerowy o predefiniowanych ruchach.
- You - informacja, że to użytkownik może wykonać ruch.
- Agent - oznacza to, że gracz komputerowy z algorytmem uczenia maszynowego ma turę w danym momencie.



Rysunek 4.4: widok czasu gry

Na obrazku znajduje się reprezentacja czasu gry w aplikacji. Każda tura trwa pewną ilość czasu, jeśli dany gracz nie zakończy swojej tury przed czasem, wszystkie jego ruchy zostają cofnięte, oraz dostaje nową płytkę z banku. Jedna tura zazwyczaj trwa 60 sekund.

Z lewej strony ekranu znajdują się pewne napisy. Są one reprezentacją listy graczy w danym trybie gry. Dostarcza również informacje, ile który gracz ma płytek na swoim stojaku. Lista graczy różni się w zależności od trybu gry:

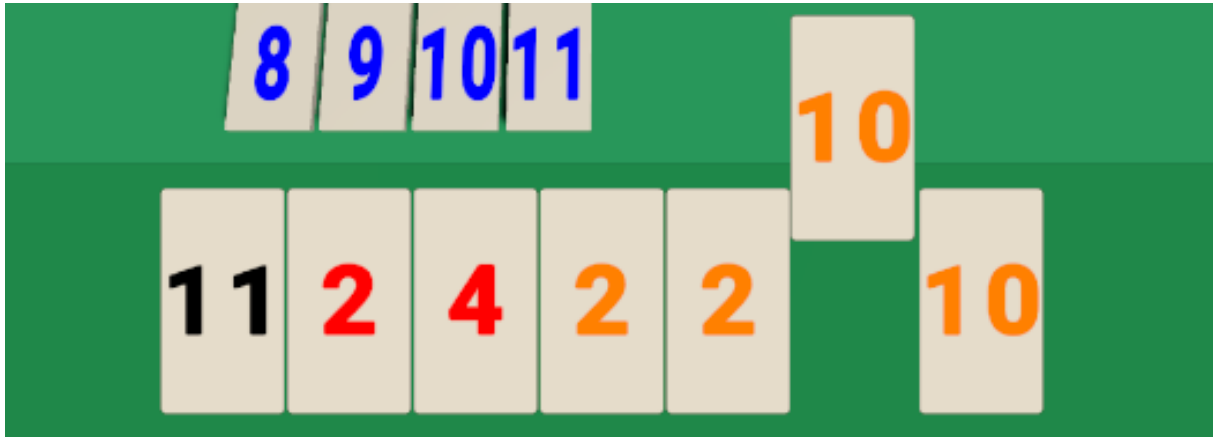


Rysunek 4.5: widok listy graczy

1. Jest to lista graczy w trybie nauki. Znajdują się tutaj trzech gracze komputerowi z predefiniowanymi ruchami oraz jeden agent. Lista w tym trybie jest jedynie poglądowa dla użytkownika.
2. To lista graczy z trybu gry użytkownik przeciwko dwóm graczom komputerowym z predefiniowanymi ruchami oraz jednemu graczowi komputerowemu z algorytmami uczenia maszynowego.
3. Jest to lista z trybu gracz przeciwko trzem graczom komputerowym z predefiniowanymi ruchami.

4. Ta lista jest z trybu gry użytkownik przeciwko gracz komputerowy z algorytmem uczenia maszynowego.

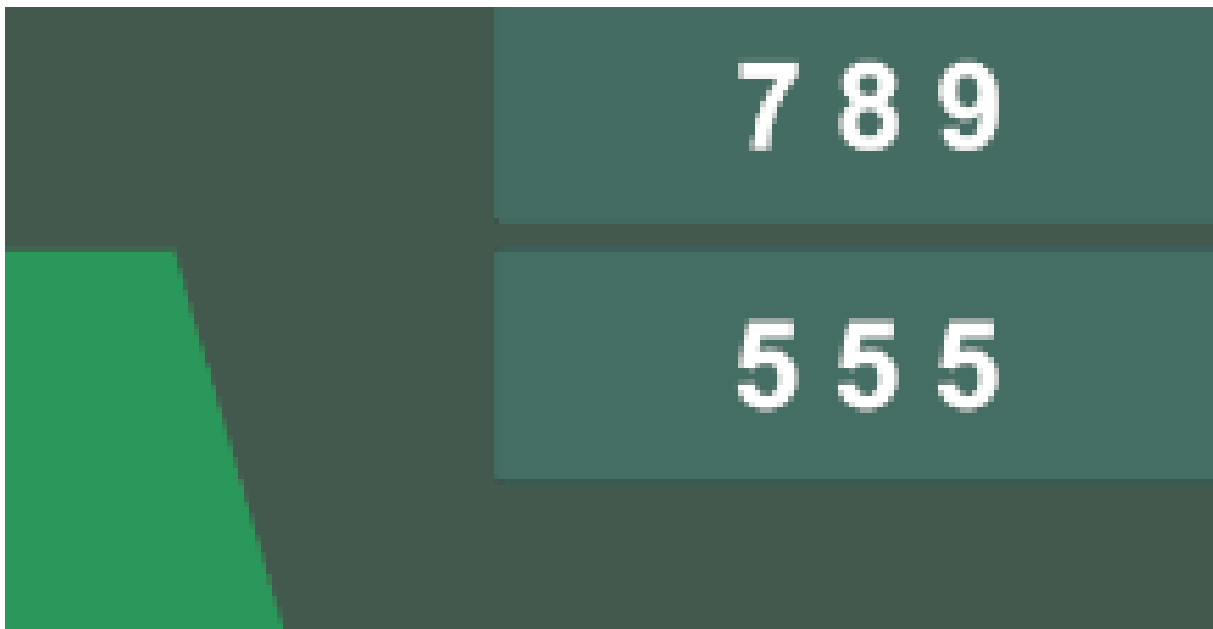
Użytkownik ma możliwość interakcji z płytkami w stojaku.



Rysunek 4.6: widok stojaku z płytkami

Przytrzymanie wybranej płytki pozwala na jej przesuwanie w prawo bądź lewo w celu ułożenia jej w konkretnej sekwencji.

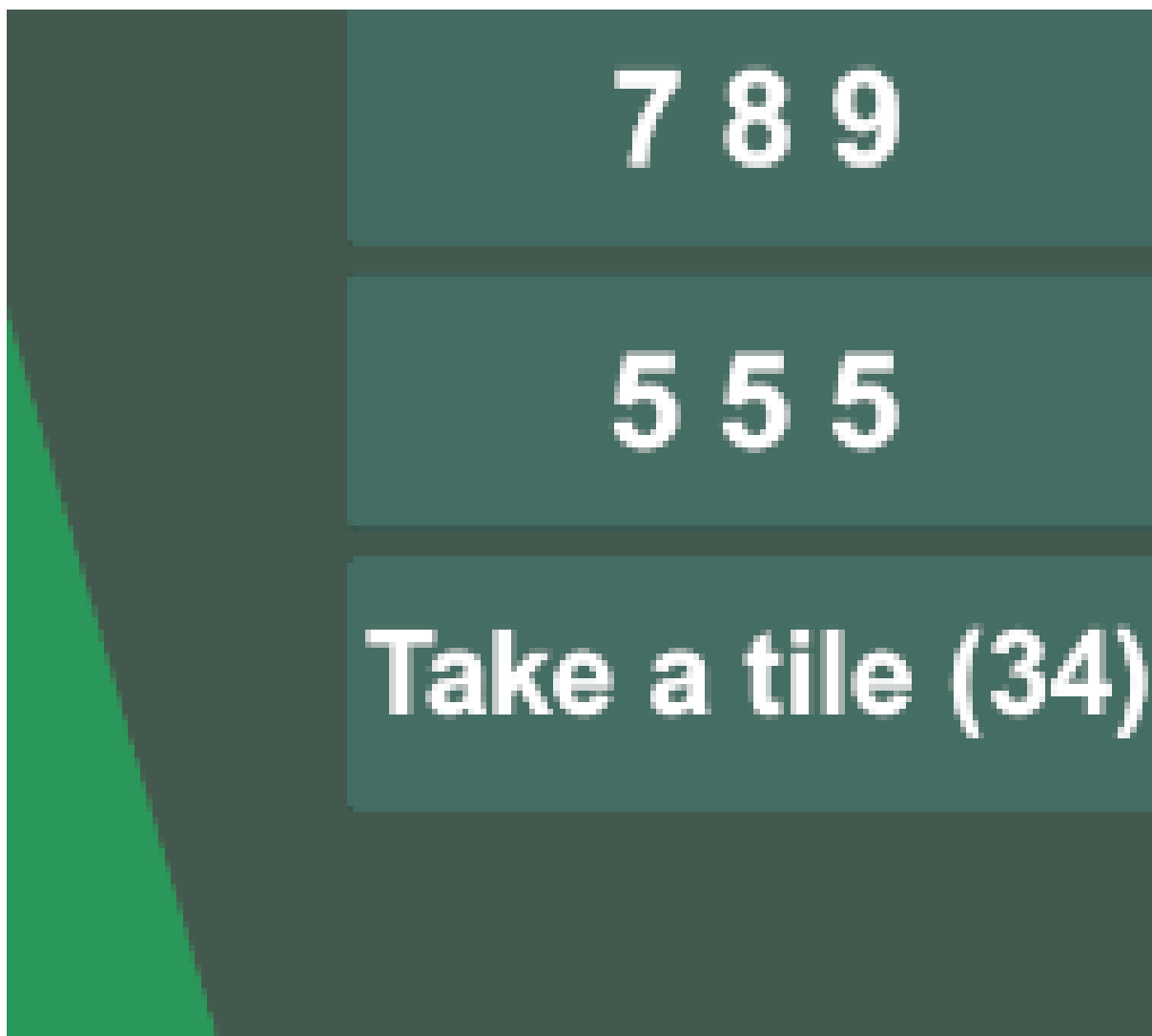
W prawym górnym rogu znajdują się przyciski, w zależności od momentu rozgrywki, są one różne:



Rysunek 4.7: przyciski poza turą

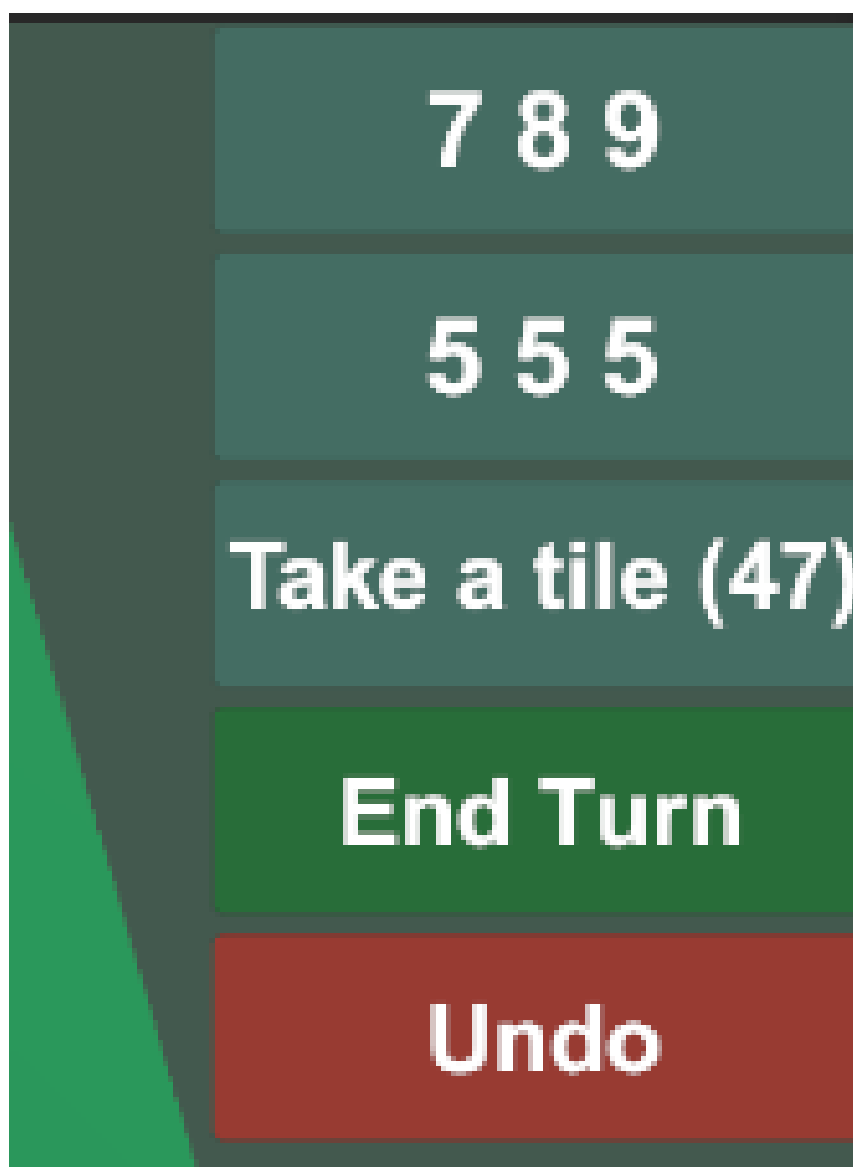
Tak wygląda widok przycisków, gdy nie ma tury użytkownika :

- pierwszy przycisk służy do sortowania względem serii.
- drugi przycisk służy do sortowania względem grup.



Rysunek 4.8: przyciski przed wykonaniem ruchu

Tak wyglądają przyciski, gdy użytkownik ma swoją turę, ale jeszcze nie wykonał ruchu. Przycisk **Take a tile (34)** pozwala graczowi na pobranie płytki, wraz z czym kończy turę. Liczba w nawiasach na zdjęciu informuje, ile zostało płytek w głównym banku. Jeśli został użyty po wykonaniu ruchu na mapie, wszystkie manipulacje zostaną cofnięte.



Rysunek 4.9: przyciski po wykonaniu ruchu

Przycisk **End Turn** wykona sprawdzenie, czy wszystkie sekwencje na mapie zawierają co najmniej trzy płytki. Jeśli funkcja wykryje błąd, tura nie zakończy się oraz zostanie pokazany komunikat o niepoprawnym zakończeniu tury.

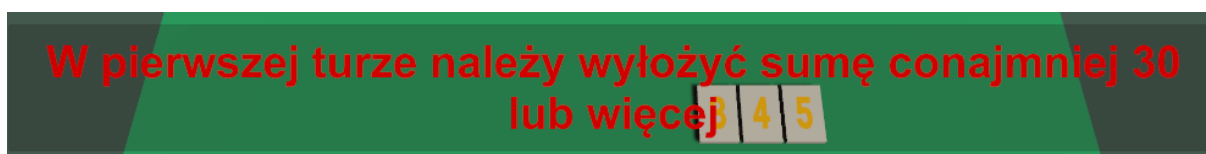
Przycisk **Undo** cofa wszystkie zmiany i wyłożenia, jakie użytkownik wykonał w trakcie danej tury.

Jeśli gracz spróbuje zakończyć turę przyciskiem **End Turn**, podczas gdy na mapie co najmniej jedna sekwencja nie zawiera minimalnie trzech płytek, na ekranie użytkownika wyświetli się ten komunikat:



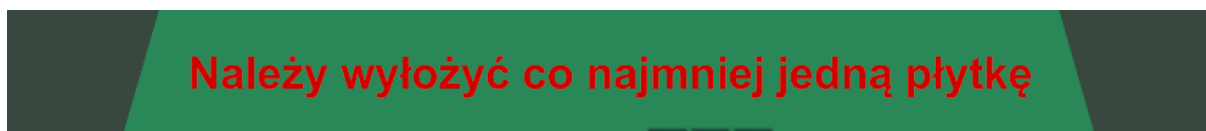
Rysunek 4.10: komunikat o źle zakończonej mapie

Jeśli gracz wykonuje swoją pierwszą turę, ale suma sekwencji, jakie wyłożył, nie jest równa bądź większa trzydzieści, to na ekranie pokaże się komunikat:



Rysunek 4.11: komunikat o złej zakończonej pierwszej turze

Gdy gracz już zakończył swoją pierwszą turę i w trakcie kolejnej tury wykona manipulacje sekwencjami na mapie nie wykładając przy tym ani jednej płytki ze swojego stojaka, pokaże się taki komunikat na mapie:



Rysunek 4.12: komunikat braku wyłożonej płytki

Nie dozwolone jest, na daną turę, manipulowanie mapą bez wyłożenia nowej płytki. Jeżeli gracz zdecyduje się zabrać z planszy płytkę, która została położona przez niego bądź kogoś innego w innej turze, pojawi się na ekranie komunikat:



Rysunek 4.13: komunikat przy niedozwolonym usuwaniu płytki

Gdy pierwszy gracz wyłoży wszystkie płytki na mapę, pokaże się ekran zakończenia gry:



Rysunek 4.14: ekran końca gry

Na zdjęciu widać hierarchię graczy w zależności od ich wyniku. Każdy gracz w hierarchii, pod zwycięzcą, ma w nawiasie zapisany wynik sumy płytek, które miał na swoim stojaku pod koniec gry. W takiej sytuacji wygrywa gracz, który nie ma już żadnych płytek na stojaku.

Jeśli gra jednak zakończy się brakiem płytek w głównym banku, ekran końca gry będzie wyglądać tak:



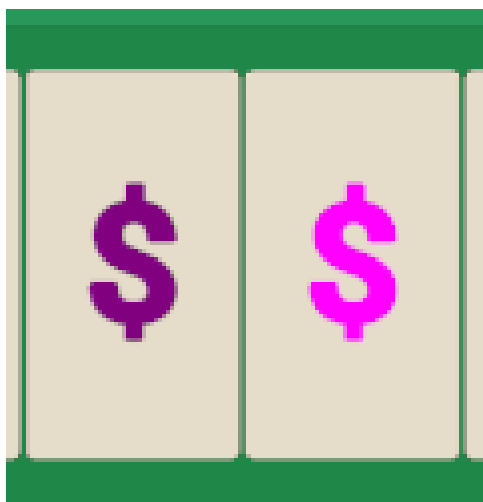
Rysunek 4.15: ekran końca gry przy pustym banku

W takiej sytuacji, każdy gracz ma wyliczoną sumę płytek na swoim stojaku. W takiej sytuacji wygrywa gracz, którego suma punktów jest najmniejsza.

Na ekranie końca gry, oprócz hierarchii, znajdują się dwa przyciski:

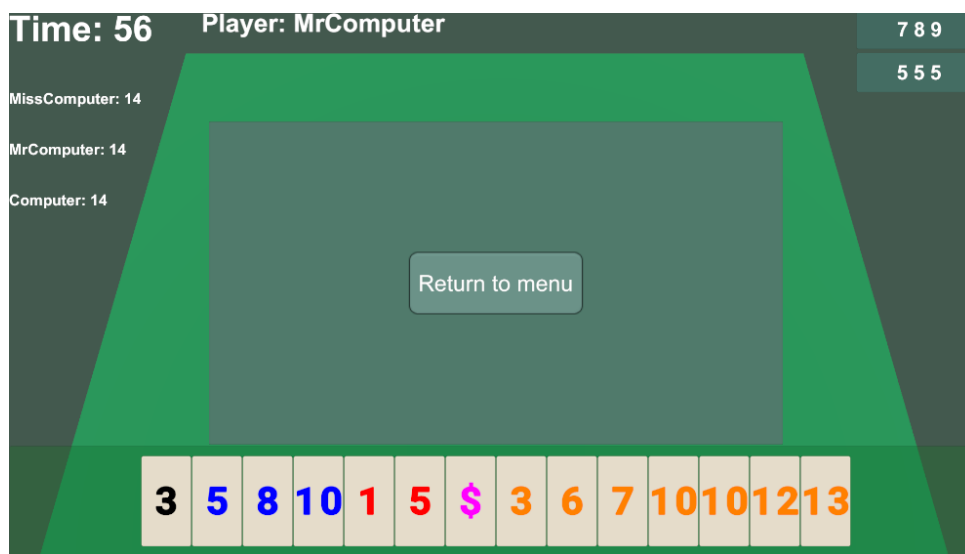
- **Return to menu** - jego użycie przeniesie użytkownika do menu głównego gry.
- **Play again** - wykorzystanie tego przycisku pozwoli graczowi wzięcie udziału w nowej rozgrywce w tym samym trybie.

Każda płytka w grze odpowiada obiektom wykorzystywanym w oryginalnej grze planszowej, oprócz płytek jokerów. Jokery w tej aplikacji wyglądają tak:



Rysunek 4.16: jokery

W trakcie gry można również ją zatrzymać, jeśli użytkownik musi na chwilę odejść od urządzenia, albo jakby chciał zmienić tryb bez przymusu czekania na zakończenie rozgrywki. W tym przypadku może nacisnąć przycisk **Esc** aby uruchomił się ekran:

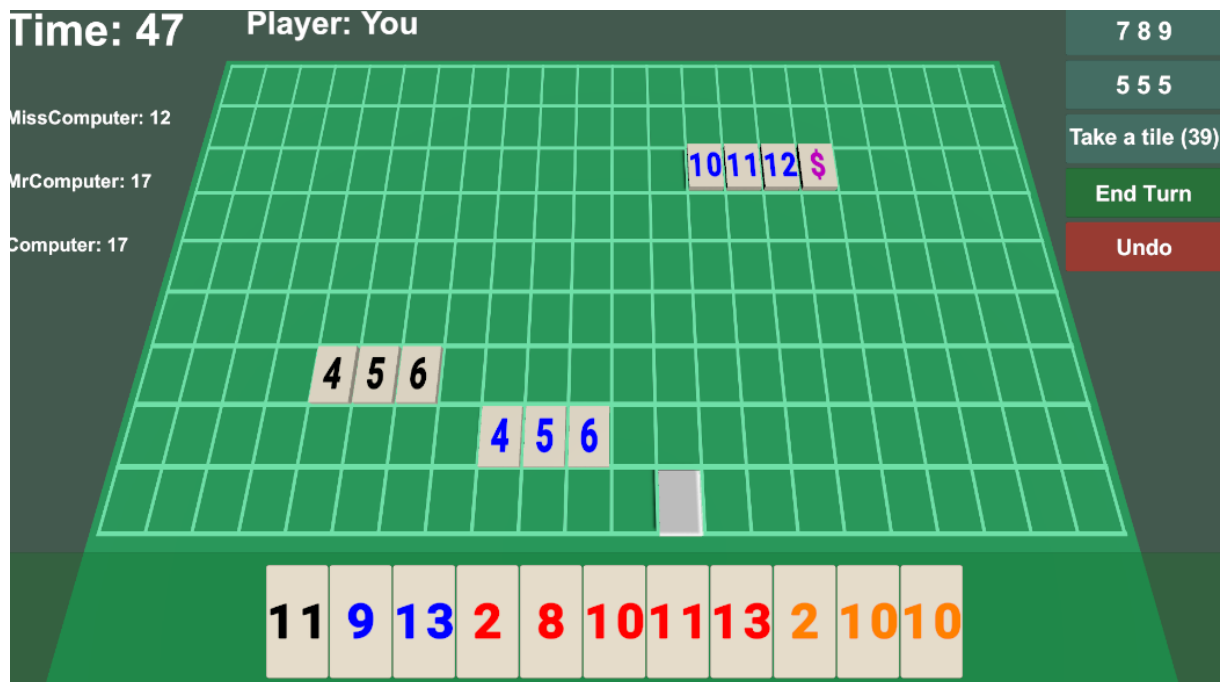


Rysunek 4.17: Ekran zatrzymania gry

W tym stanie, gra zostaje zatrzymana. Wciśnięcie przycisku **Return to menu** będzie skutkowało przedwczesnym zakończeniem gry i powrotem do menu głównego. Aby wznowić grę, użytkownik musi ponownie wcisnąć klawisz **Esc**.

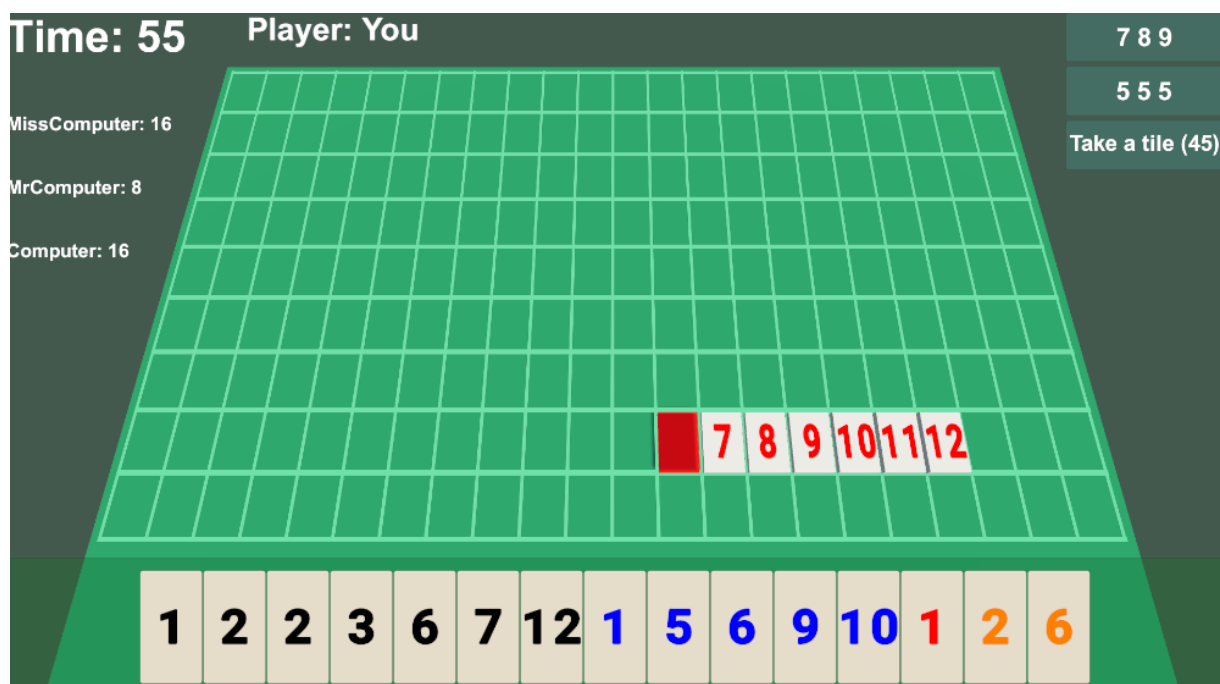
4.4.3 Rozgrywka

Po wybraniu trybu gry, użytkownik widzi mapę rozgrywki. Zadaniem gracza jest, aby jak najszybciej wyłożyć płytki ze swojego stojaka. Podczas interakcji z mapą, pojawia się siatka, aby gracz wiedział, gdzie może płytka być położona. Aby położyć płytkę na mapie, należy nacisnąć lewym przyciskiem myszy wybraną płytkę, a następnie przesunąć kursor myszy w stronę wybranej pozycji na mapie.



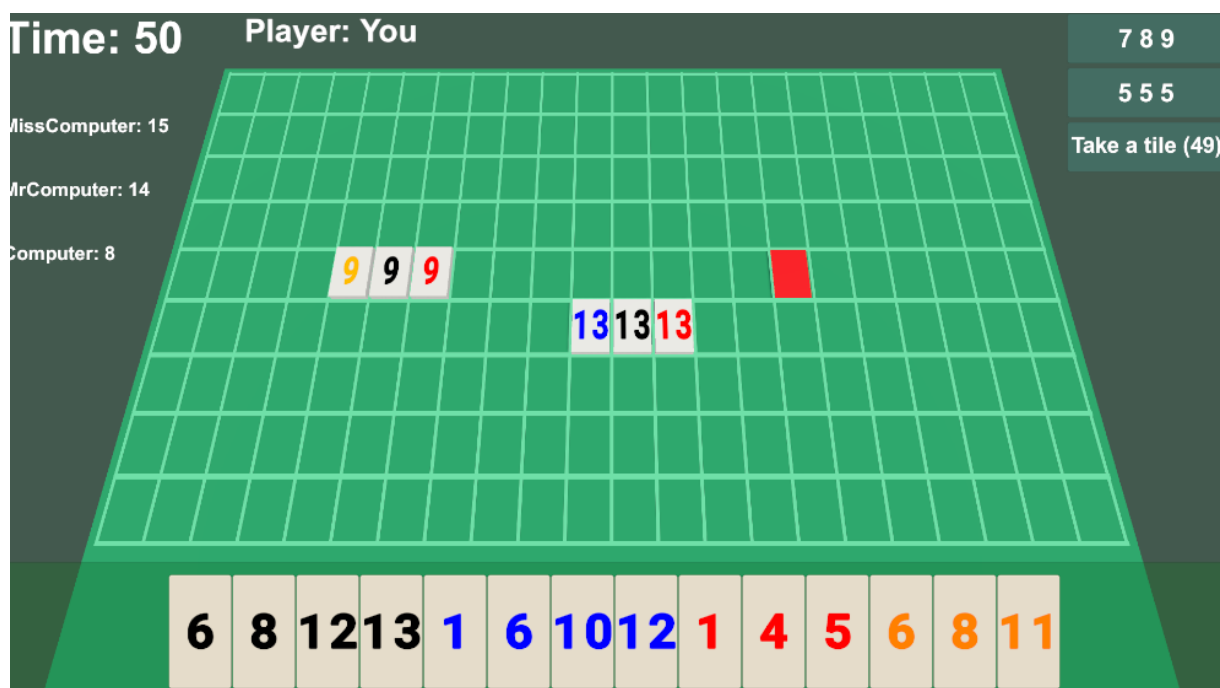
Rysunek 4.18: Kładzenie płytki na mapę

Na ekranie widać obiekt, podobny do płytki. Gracz dzięki temu widzi, gdzie płytka będzie położona. Aby zakończyć akcję, wystarczy nacisnąć lewym przyciskiem myszy punkt na mapie. Jeśli gracz przeciągnie kursorem myszy na pole, na którym nie może położyć płytki, obiekt zostanie pokazany na czerwono



Rysunek 4.19: Złe położenie płytki

Aby usunąć płytkę z mapy, należy wcisnąć klawisz D, a następnie kliknąć lewym przyciskiem myszy na płytkę, którą chce się usunąć. Ta akcja usuwa płytkę z mapy i przywraca ją do stojaka gracza. Użytkownik może usunąć jedynie płytki wyłożone w trakcie trwającej tury. Gracz, chcąc usunąć płytkę z mapy zobaczy czerwony obiekt jako kursor. Oznacza on, że nie można usunąć obiekt z miejsca, w którym obiektu nie ma. Obiekt zmienia kolor na biały, gdy zostanie umiejscowiony na pozycji, gdzie znajduje się płytki



Rysunek 4.20: wygląd usuwania płytki

Gdy gracz chce przesunąć płytkę na mapie, musi najpierw nacisnąć prawym przyciskiem myszy na wybraną płytkę. Następnie wybrać nową pozycję na mapie i nacisnąć lewy przycisk myszy na wybranej pozycji. W trakcie przesuwania płytki pojawia się obiekt tak samo, jak przy wykładaniu nowej płytki

Jeśli gracz zdecyduje się zaprzestać wykonywania którejś akcji jak wykładania czy przesuwania, musi nacisnąć klawisz Q

Rozdział 5

Specyfikacja wewnętrzna

Specyfikacja wewnętrzna opisuje szczegóły techniczne implementacji projektu. Projekt został zrealizowany w środowisku Unity, jego kluczowe komponenty zostały zaimplementowane z C#. To pozwoliło zaimplementować logikę gry, obiekty wykorzystywane w grze oraz gracza komputerowego z algorytmami uczenia maszynowego.

5.1 Klasa Tile

Klasa Tile jest wykorzystywana do implementacji pojedynczej płytki w grze Rummi-kub. Każda płytki ma swój numer i kolor, również jokery w tym projekcie mają swoje konkretne kolory i wartości. Klasa zawiera cztery zmienne:

```
1      private int number;  
2      private UnityEngine.Color numberColor;  
3      private string tilename;  
4      private string symbol;  
5      private bool put;
```

- number - określa wartość danej płytki, joker ma wartość 30.
- numberColor - klasa środowiska UnityEngine, określa kolor dla płytki w systemie RGB.
- tilename - nazwa płytki wykorzystywana do nazywania obiektów tworzonych na mapie.
- symbol - zmienna do określania płytki, wykorzystywana przy inicjowaniu obiektu na mapie, aby użytkownik widział, jaką płytkę widzi. Taka zmienna jest konieczna przez istnienie jokerów w grze, którzy na mapie są widoczni jako "\$"

- put - określa, czy dana płytką może zostać usunięta z mapy. Jeśli zmienna jest równa `False`, wtedy można usunąć, jeśli `True` co oznacza, że płytką została położona i nie można jej usunąć.

5.2 Struktura danych

Aplikacja musi być w środowisku 2.5D, co wymaga zaimplementowania obiektów w przestrzeni 3D.

Stojak gracza jest obiektem 2D, każda płytką, jaką gracz posiada, jest obiektem typu `photo` z dynamicznie zmiennym tekstem. W pamięci, stojak jest przechowywany jako lista obiektów klasy `Tile`.

Jednak sama mapa, jak i obiekty na mapie, są zaimplementowane w przestrzeni 3D. W pamięci systemu, mapa jest reprezentowana jako słownik płytek `Dictionary<Vector3Int, Tile>`, gdzie każda lokalna pozycja na mapie jest zapisana jako klucz. Mapa, widziana przez użytkownika, opiera się na funkcjach siatki, jest to wbudowana funkcjonalność środowiska Unity. Siatka składa się z wierszy i kolumn, która pozwala na jednoznaczne określenie pozycji obiektów na mapie. Najważniejsze funkcjonalności wykorzystane w projekcie:

1	<code>CellToWorld (Vector3Int)</code>
2	<code>WorldToCell (Vector3)</code>

- `CellToWorld` - Wartością wejściową funkcji jest Wektor 3D z wartościami całkowitymi, zwraca natomiast Wektor 3D wartości zmiennoprzecinkowych. Pozwala na wyciągnięcie pozycji obiektu w przestrzeni globalnej w czasie interakcji z nimi.
- `WorldToCell` - funkcja interpretująca pozycje świata obiektu do pozycji lokalnej siatki. Zmienną wejściową do funkcji jest Wektor 3D z wartościami zmiennoprzecinkowymi. Funkcja zwraca Wektor 3D liczb całkowitych

Wykorzystanie tych funkcji pozwala na łatwiejsze przechowywanie danych i pozycji na planszy.

5.3 System przemieszczania

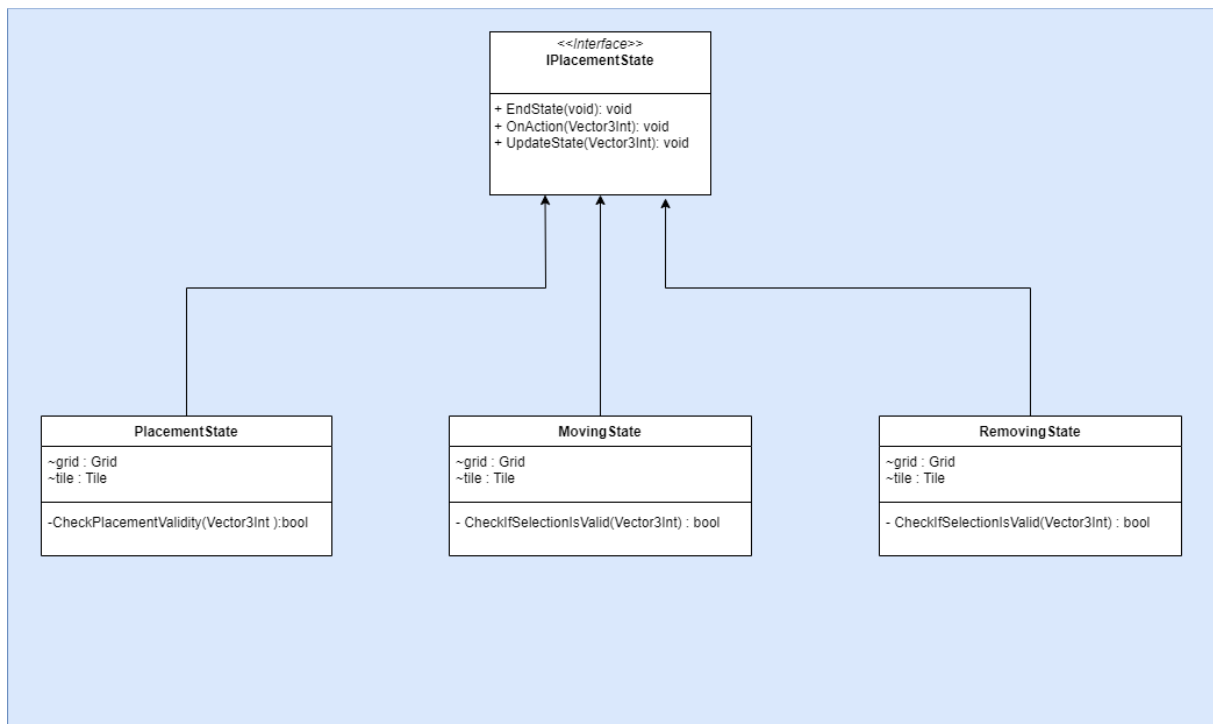
System przemieszczania w aplikacji opiera się na interfejsie `IPlacementState`, który definiuje wspólny zestaw funkcji umożliwiających zarządzanie obiektami na mapie siatki (grid). Interfejs został zaimplementowany przez trzy klasy: `PlacementState`, `RemovingState` oraz `MovingState`, które różnią się sposobem inicjowania i realizacji funkcji.

```

1  public interface IPlacementState
2  {
3      void EndState();
4      void UpdateState(Vector3Int gridPosition);
5      void OnAction(Vector3Int gridPosition);
6  }

```

Każda interakcja użytkownika z obiektem tworzy nowy obiekt `IPlacementState`. Gdy gracz zakończy wybraną akcję, zaimplementowany interfejs zostaje usunięty.



Rysunek 5.1: Uproszczony diagram IPlacement

5.4 Biblioteka ML-Agents

Unity ML-Agents Toolkit to biblioteka służąca do implementowania i trenowania agentów z algorytmami uczenia maszynowego. Biblioteka pozwala na wykorzystanie metod takich jak uczenie ze wzmocnieniem, uczenie nadzorowane oraz uczenie w grupie. Biblioteka ML-Agents została wybrana do implementacji przeciwnika komputerowego w grze Rummikub, ponieważ:

- Umożliwia łatwe tworzenie środowiska treningowego w Unity.
- Pozwala dostosowywać metody modelu bezpośrednio do mechanik gry.
- Daje możliwość modyfikacji zmiennych modelu.

Kluczowymi komponentami ML-Agents są obserwacje, akcje oraz nagrody.

1. Obserwacje dają agentowi potrzebne informacje, aby zaplanować swoje kolejne ruchy. Ilość obserwacji jest ustalana przy implementacji statycznie, nie ma możliwości zmieniać ilości obserwacji w trakcie rozgrywki.
2. Akcje są funkcjami, jakie podejmuje na bazie obserwacji oraz nagród, jakie otrzymuje. Agent podejmuje akcje, dzielą się one na dwa typy
 - akcje ciągłe - model wyznacza akcje z podanego przedziału liczbowego w wartościach zmiennoprzecinkowych, zazwyczaj od -1 do 1. Takie akcje są najlepsze przy funkcjach ruchu czy rotacji.
 - akcje dyskretne - akcje modelu z podanego przedziału liczbowego w wartościach całkowitych. Najlepsze do konkretnych decyzji, jak wybieranie indeksów płytek w liście.
3. Otrzymywanie nagród i kar za konkretne działania dają agentowi informacje, jakie ruchy powinien wykonywać a jakie nie.

Decyzje agenta są interpretowane jako wartości liczbowe, przy implementacji należy przypisać konkretnych wartościom funkcje. W zależności od decyzji agenta, należy mu przypisać nagrodę bądź karę. Z czasem agent uczy się, jakie decyzje są najbardziej opłacalne i częściej je wykonuje.

Zmienne modelu mogą być modyfikowane w plikach typu `.yaml`. Na stronie [4] jest to dokładniej opisane.

```

1 behaviors:
2   behaviors_name:
3     trainer_type: ppo
4     hyperparameters:
5       batch_size: 64
6       buffer_size: 2048
7       learning_rate: 3.0e-4
8       beta: 5.0e-4
9       epsilon: 0.2
10      lambd: 0.95
11      num_epoch: 3
12      learning_rate_schedule: linear
13      beta_schedule: constant
14      epsilon_schedule: linear
15    network_settings:
16      normalize: true
17      hidden_units: 128
18      num_layers: 2
19    reward_signals:
20      extrinsic:
21        gamma: 0.99
22        strength: 1.0
23    max_steps: 500000
24    time_horizon: 64
25    summary_freq: 10000

```

Rysunek 5.2: Domyślne środowisko .yaml .

Na zdjęciu widać domyślną konfigurację. Modyfikacja parametrów środowiska pozwala na dostosowanie modelu do aplikacji, w której jest implementowane.

Każdy parametr odpowiada innej funkcjonalności modelu:

- **behaviors** - jest główną sekcją definiującą zachowanie agenta. Zawiera wszystkie szczegóły dotyczące jego treningu.
- **behaviors_name** - nazwa konkretnego zachowania, musi być identyczna do zaimplementowanej nazwy w Unity.
- **trainer_type** - Typ algorytmu używanego do treningu. W przykładzie zawarty jest algorytm ppo (Proximal Policy Optimization).
- **hyperparameters** - przedział zmiennych algorytmu:

1. `batch_size` - liczba próbek przetwarzanych w jednej iteracji.
 2. `buffer_size` - liczba przechowywanych danych w pamięci bufora przez aktualizacją sieci.
 3. `learning_rate` - określone tempo aktualizacji wag modelu podczas każdej iteracji.
 4. `beta` - współczynnik entropii, im wyższa wartość, tym model jest bardziej skłonny do eksploracji nowych strategii podczas nauki.
 5. `epsilon` - wyznacza maksymalną wartość, z jaką może zmieniać się polityka (policy) w jednej aktualizacji.
 6. `lambda` - współczynnik wygaśnięcia, do balansowania między dokładnością a stabilnością wyceny nagrody.
 7. `num_epoch` - liczba iteracji gradientowych dla każdej próbki.
 8. `learning_rate_schedule`, `beta_schedule`, `epsilon_schedule` - określają jak wartości zmiennych `learning_rate`, `beta` oraz `epsilon` zmieniają się w czasie treningu.
- **`network_settings`** - definiuje strukturę sieci neuronalnej, której używa agent:
 1. `normalize` - definiuje, czy dane wejściowe mają zostać zoptymalizowane czy nie.
 2. `hidden_units` - liczba jednostek w każdej warstwie ukrytej sieci neuronalnej.
 3. `num_layers` - ilość warstw sieci neuronalnej.
 - **`reward_signals`** - definiuje sygnał nagrody dla agenta. Tutaj jest użyty typ `extrinsic`, czyli nagroda bazująca na środowisku:
 1. `gamma` - współczynnik wygaśnięcia, który określa w jakim stopniu agent bierze pod uwagę przyszłe nagrody.
 2. `strength` - skala wpływu sygnału na trening.
 - **`max_steps`** - Maksymalna liczba kroków treningowych, jakie może wykonać agent. Po osiągnięciu tej wartości, model nie może być dalej uczony.
 - **`time_horizon`** - maksymalny czas, po jakim agent kończy zbieranie danych do bufora.
 - **`summary_freq`** - określa, co ile zostają wygenerowane dane podsumowania treningu.

Parametry środowiskowe mogą się różnić w zależności od wybranego `trainer_type`. Przy grze Rummikub ważne jest, aby dostosować parametry do dużej ilości obserwacji oraz akcji.

5.5 Klasa **PlayerIA**

Klasa **PlayerIA** implementuje metody dla agenta do wykonywania konkretnych funkcjonalności i interakcji w środowisku. Klasa dziedziczy po interfejsie **Agent**, która jest dostarczana przez bibliotekę ML-Agents.

Rozdział 6

Weryfikacja i walidacja

6.1 Testowanie

6.2 Wykryte błędy

Rozdział 7

Podsumowanie i wnioski

7.1 Napotkane problemy

7.2 Możliwy rozwój

Bibliografia

- [1] Python Software Foundation. *Beginners Guide to Python: Download*. 2024. URL: <https://wiki.python.org/moin/BeginnersGuide/Download> (term. wiz. 05.11.2024).
- [2] Rummikub. *Zasady gry Rummikub*. PDF dokument. `rummikub_rules.pdf`. 2019.
- [3] Unity Technologies. *Installing Unity*. 2016. URL: <https://docs.unity3d.com/550/Documentation/Manual/InstallingUnity.html> (term. wiz. 11.12.2024).
- [4] Unity Technologies. *ML-Agents Toolkit: Creating New Learning Environments*. 2024. URL: <https://github.com/Unity-Technologies/ml-agents/blob/develop/docs/Learning-Environment-Create-New.md> (term. wiz. 15.12.2024).
- [5] Unity Technologies. *ML-Agents Toolkit: Installation Guide*. 2024. URL: <https://github.com/Unity-Technologies/ml-agents/blob/develop/docs/Installation.md> (term. wiz. 11.12.2024).
- [6] Unity Technologies. *Unity System Requirements*. 2023. URL: <https://docs.unity3d.com/2022.3/Documentation/Manual/system-requirements.html> (term. wiz. 10.12.2024).

Dodatki

Spis skrótów i symboli

Źródła

Lista dodatkowych plików, uzupełniających tekst pracy

Spis rysunków

3.1	Diagram przypadków użycia	9
4.1	Menu gry	16
4.2	Przykładowy ekran początku gry	17
4.3	Aktualna tura: Gracz X	17
4.4	widok czasu gry	18
4.5	widok listy graczy	18
4.6	widok stojaku z płytkami	19
4.7	przyciski poza turą	19
4.8	przyciski przed wykonaniem ruchu	20
4.9	przyciski po wykonaniu ruchu	21
4.10	komunikat o źle zakończonej mapie	22
4.11	komunikat o źle zakończonej pierwszej turze	22
4.12	komunikat braku wyłożonej płytki	22
4.13	komunikat przy niedozwolonym usuwaniu płytki	22
4.14	ekran końca gry	23
4.15	ekran końca gry przy pustym banku	23
4.16	jokery	24
4.17	Ekran zatrzymania gry	24
4.18	Kładzenie płytki na mapę	25
4.19	Złe położenie płytki	26
4.20	wygląd usuwania płytki	26
5.1	Uproszczony diagram IPlacement	31
5.2	Domyślne środowisko .yaml	33

Spis tabel